(Uri) 1. The operation uses SAL or shift left. It left shifts a byte, or word specified by an immediate value and stores the product in that byte.

(Uri) 2.

CODE Produced

```
.LFB1:
    .cfi_startproc
    endbr64
    pushq    %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    movl    %edi, -4(%rbp)
    movl    -4(%rbp), %edx
    movl    %edx, %eax
    sall    $2, %eax
    addl    %edx, %eax
    sall    $3, %eax
    addl    %edx, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
```

PSEUDOCODE

%eax = x
%edx = x
sall $2 %eax
%eax = x * 2^2
%eax = 4x
addl %edx, %eax
%eax = x + 4x
%eax = 5x
SAL $3 %eax
%eax = 5x << 3
%eax = 5x * 2^3
%eax = 40x
ADDL %edx %eax
%eax = x + 40x
%eax = 41x
%eax = x * 41

(Uri) 3. **Code Produced**

```
.cfi_startproc
   endbr64
   pushq    %rbp
   .cfi_def_cfa_offset 16
   .cfi_offset 6, -16
   movq    %rsp, %rbp
   .cfi_def_cfa_register 6
   movl    %edi, -4(%rbp)
   movl    -4(%rbp), %edx
   movl    %edx, %eax
   sall    $6, %eax
   subl    %edx, %eax
   popq    %rbp
   .cfi_def_cfa 7, 8
   ret
   .cfi_endproc
```

Pseudocode
%eax = x
%edx = x
SAL $6 %eax
%eax = x * 2^6
%eax = 64x
SUBL %edx, %eax
%eax = 64x - x
%eax = 63x
%eax = x * 63

(Dave) 4.
Generated Assembly Code:

```
.LFB3:
        .cfi_startproc
        endbr64
        pushq %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq  %rsp, %rbp
        .cfi_def_cfa_register 6
        movl   %edi, -4(%rbp)
        movl   -4(%rbp), %edx
        movl   %edx, %eax
        sall    $2, %eax
        addl    %edx, %eax
```

```
    negl    %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
```

Proof:

movl %edx, %eax
- This line initializes the eax register with the value of edx which is x.

sall $2, %eax
- Then, the eax register is shifted to the left by 2 which is equivalent to multipying x by 2^2 (4).
- The current value of eax is 4x.

addl %edx, %eax
- Then, it adds x (stored in edx) to 4x (stored in eax).
- The current value of eax is 5x.

negl %eax
- Lastly, we need to negate the value of eax which is equivalent to subtracting 5x from 0.
- The final value of eax is -5x.

Pseudo-code (C++):

Input x is in register edx.

```
// movl %edx, %eax
// output == x
output = x;

// sall   $2, %eax
// output == x * 2^2 == 4x
output = output << 2;

// addl  %edx, %eax
// output == 4x + x
// output == 5x
output = output + x;

// negl  %eax
// output == -(5x)
// output == -5x
output = -output;
```

(Dave) 5.

Generated Assembly Code:

```
.LFB4:
```

```
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq  %rsp, %rbp
.cfi_def_cfa_register 6
movl   %edi, -4(%rbp)
movl   -4(%rbp), %eax
imull   $61, %eax, %eax
popq   %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
```

The unexpected behavior is that instead of using arithmetic shifts, x * 61 is multiplied directly using the imul. A single imul instruction (e.g., x * 61 = 61x) is more compact and faster, but a series of arithmetic shifts and additions can still be used (e.g., x * 64 - x * 3, but x * 3 = x * 2 + x or x * 3 = x * 4 - x). The "unexpected behavior" occurs because the compiler's cost analysis model determined that a single imul instruction is more efficient than the equivalent sequence of shifting, addition, and subtraction, which exceeds a modern imul instruction's latency (typically 3 clock cycles). As a result, direct multiplication produces faster and more compact code.

Source:
https://superuser.com/questions/643442/latency-of-cpu-instructions-on-x86-and-x64-processors
https://stackoverflow.com/questions/37925143/x86-64-is-imul-faster-than-2x-shl-2x-add#:~:text=4-,5,20%2C%202016%20at%2014:35

(Jaren) 6.

With the -O flag, the compiler reduced the number of instructions by using ‘lea’ (Load Effective Address) instruction. While ‘lea’ is designed to calculate memory addresses, the compiler uses it to perform arithmetic calculations because it can handle addition and multiplication (by 1, 2, 4, or 8) in a single step.

The standard ALU instructions (add, sub, sal) perform one operation at a time. On the other hand, the ‘leal’ instruction performs a calculation in the form of:
Result = Base + (Index x Scale), wherein the scale can be 1, 2 ,4, or 8

For example, on dummy2: x times 41. Instead of shifting and adding repeatedly, the optimized assembly does x times 41 in just two instructions:

1. **leal (%rdi,%rdi,4), %eax**
    a. This uses %rdi (which holds x) as both the base and the index.
    b. It calculates x + (x times 4), resulting in 5x to be stored at %eax.

2. **leal (%rdi,%rax,8) %eax**
    a. This uses %rdi (x) as the base and the previous result %rax (5x) as the index.
    b. It calculates x + (5x times 8), resulting in x + 40x = 41x.

In hindsight, the optimization collapses a chain for dependent instructions (shifts and adds) into just 2 ‘leal’ instructions. To further, modern CPUs actually have a dedicated Address Generation Unit (AGU) which is separate from ALU. By using ‘leal’, the CPU can process calculations on the AGU while ALU can handle other tasks simultaneously.

Standard math instructions (like add or sub) automatically update the CPU’s status flags (EFLAGS). The CPU often has to wait for these updates to finish before moving on. The ‘leal’ instruction does not touch these flags, allowing the CPU to execute the code immediately without waiting, which prevents stalling in the execution pipeline.

References:

- https://youtu.be/7WrSaM7pcWY
- https://handmade.network/wiki/7111-using_the_lea_instruction_for_arbitrary_arithmetic
- https://stackoverflow.com/questions/6323027/lea-or-add-instruction
- https://www.oreateai.com/blog/understanding-the-lea-instruction-in-assembly-language/2ef0ccf82b4aa1dd64872a95a0deb001